Real Time Enbedded System
Project report

Studnet :
Maude BANCHELIN

Level :
Aéro 4 Promotion 2027

28 april 2026
School year 2025-2026
M. SINGH

Introduction

Real-time scheduling is a critical component that ensures every task is executed within a strict timeframe. Failure to meet a deadline can lead to mission failure or critical safety issues.

This report presents a comprehensive study of a non-preemptive real-time system composed of seven periodic tasks. The primary objective is to develop a scheduling strategy that maximizes processor efficiency while maintaining system stability. A key focus of this project is the experimental determination of the Worst-Case Execution Time (WCET) for a specific computation task (τ_1).

Throughout this study, we will evaluate the system's schedulability using the Earliest Deadline First (EDF) algorithm. Furthermore, we will compare a standard scheduling scenario with an optimized version where a specific task (τ_5) is allowed to miss its deadline.

Contents

1 Tasks schedulability 1
2 Methodology 1
2.1 WCET computation 1
2.2 Non-preemptive algorithm 3
3 Scheduling results 4
3.1 Scenario 1: Standard NP-EDF (All tasks on time) 5
3.2 Scenario 2: τ_5 is allowed to miss its deadline 5
3.3 Response time analysis results 6
3.4 Comparative analysis 8
4 Computational complexity 9
5 Conclusion 10

1 Tasks schedulability

The tasks to schedule is :

Task	C	T_i
τ_1	C_1	10
τ_2	3	10
τ_3	2	20
τ_4	2	20
τ_5	2	40
τ_6	2	40
τ_7	3	80

Table 1: Task set

First, we need to test the schedulability of our set.

$$\frac{C_1}{10} + \frac{3}{10} + \frac{2}{20} + \frac{2}{20} + \frac{2}{40} + \frac{2}{40} + \frac{3}{80} < 1 \tag{1}$$

$$\iff C_1 < 3.625ms \tag{2}$$

We determined the hyperperiod as the lowest common multiple of the periods T_i :

$$Hyperperiod = LCM(10, 20, 40, 80) = 80$$

Finnaly, we have the hyperperiod $= 80$ and $C_1 < 3.625$ for the set to be schedulable.

2 Methodology

2.1 WCET computation

To determine a value of C_1, we computed the WCET. I decided to code my project under `python` but the first task, τ_1, involves intensive arithmetic operations (multiplication of large numbers). To ensure a reliable estimation of the Worst-Case Execution Time (C_1), I combined the multiplication in `C` code and the rest of the WCET in `python`. This allowed me to obtain a more efficient code.

2.1.1 Technical Approach

The multiplication logic was implemented in C. This function, `multiply_logic`, handles 64-bit unsigned integers to provide a stable execution baseline.

The integration is performed using the `ctypes` library, allowing Python to call the compiled C shared library directly.

2.1.2 Experimental setup and commands

The measurements were conducted within a Windows Subsystem for Linux (WSL) environment. The following commands are used to generate the results:

WSL Execution Commands

```
# Compile the C library
gcc -fPIC -shared -o multiplication.so multiplication.c

# Run the statistical analysis script
python3 WCET_BANCHELIN_Maude.py
```

2.1.3 Statistical analysis

We performed $50,000$ iterations to capture a wide distribution of execution times. This large sample size allows us to filter out OS-level interrupts and identify the true Worst-Case. The script calculates the following metrics:

- **Minimum and Maximum (WCET):** To define the execution boundaries.
- **Quartiles (Q_1, Q_2, Q_3):** To understand the distribution and stability of the task.
- **Safety Margin:** A 20% margin is added to the observed maximum to define the final C_1 used in the scheduler, accounting for potential unforeseen hardware delays.

2.1.4 WCET results

After a few compilations, the Worst-Case is :

```
--- Statistics for Tau 1 (via C) ---
Min:   0.000386 ms
Max (WCET C1): 2.733153 ms
Max with 20% margin (WCET C1): 3.279784 ms
Q1:    0.000412 ms
Q2:    0.000428 ms
Q3:    0.000505 ms
```

Figure 1: WCET Result

We have $C_1 = 3.279784$ms wich respect perfectly our schedulability condition ($C_1 < 3.625$).

So, until the end of the project, I will use $C_1 = 3.279784.$ ms

With this value, we have :

$$\frac{3.279784}{10} + \frac{3}{10} + \frac{2}{20} + \frac{2}{20} + \frac{2}{40} + \frac{2}{40} + \frac{3}{80} = 0.96548$$

It represents the CPU Utilization ($U = 96.548\%$).

2.2 Non-preemptive algorithm

To schedule the task set, I implemented a Non-Preemptive Earliest Deadline First (NP-EDF) algorithm. I chose this algorithm because we needed to maximize processor idle time while ensuring all critical tasks meet their respective deadlines.

2.2.1 Algorithm logic and priority

The algorithm operates at the job level across a total hyperperiod of 80 ms. At any given time t, the scheduler examines the queue of ready jobs (jobs whose arrival time $a_{ij}^n \leq t$) and selects the one with the closest absolute deadline.

Unlike standard EDF, this implementation is non-preemptive. Indeed, once a job starts its execution, it occupies the processor for its full duration C_i without interruption. This approach is efficient for real-time embedded systems as it eliminates context-switching overhead and simplifies the verification of response times.

2.2.2 Handling task τ_5 deadline exception

According to the assignment constraints, I developped a specific version of the schedule where task τ_5 is allowed to miss its deadline to minimize the total waiting time of the system. We will compare the 2 versions later.

To implement this, I modified the selection logic to deprioritize τ_5 :

- The scheduler first checks for any ready jobs that are not τ_5.
- If such jobs exist, the one with the earliest deadline is prioritized.
- τ_5 is only selected for execution if no other higher-priority tasks are ready, or if its own priority is artificially elevated when no other choice remains.

This deprioritization of τ_5 ensures that more critical tasks with shorter periods (like τ_1 and τ_2) are not delayed by τ_5, thus reducing the overall system latency.

On my Github repository, you will find two codes. One where τ_5 can miss its dealine and the other where it cannot.

2.2.3 Response time verification

To ensure the schedulability of the system, the response time R_{ij}^n for each job is calculated and verified against its deadline. The formula used for this verification, as provided in the assignment, is:

$$R_{ij}^n = C_{ij}^n + \max(R_{ij}^{n-1} - (a_{ij}^n - a_{ij}^{n-1}), 0) \tag{3}$$

This allows us to confirm that for all tasks, $R_{ij}^n \leq T_i$ is satisfied.

3 Scheduling results

This section presents the results obtained from the two different scheduling scenarios. The objective is to evaluate the impact of task prioritization on the overall system performance, specifically regarding the total waiting time and the respect of deadlines.

3.1 Scenario 1: Standard NP-EDF (All tasks on time)

In this first scenario, the algorithm follows a strict Non-Preemptive Earliest Deadline First (NP-EDF) logic. Every task instance is treated according to its absolute deadline.

Jobs generated for the hyperperiod of 80 ms : 29 jobs

	Task	Instance	Start	End	Deadline	Wait_Time	idle_Time	Status
0	T1, J1	1	0.0000	3.2798	10	0.0000	0	SUCCESS
1	T2, J1	1	3.2798	6.2798	10	3.2798	0	SUCCESS
2	T3, J1	1	6.2798	8.2798	20	6.2798	0	SUCCESS
3	T4, J1	1	8.2798	10.2798	20	8.2798	0	SUCCESS
4	T1, J2	2	10.2798	13.5596	20	0.2798	0	SUCCESS
5	T2, J2	2	13.5596	16.5596	20	3.5596	0	SUCCESS
6	T5, J1	1	16.5596	18.5596	40	16.5596	0	SUCCESS
7	T6, J1	1	18.5596	20.5596	40	18.5596	0	SUCCESS
8	T1, J3	3	20.5596	23.8394	30	0.5596	0	SUCCESS
9	T2, J3	3	23.8394	26.8394	30	3.8394	0	SUCCESS
10	T3, J2	2	26.8394	28.8394	40	6.8394	0	SUCCESS
11	T4, J2	2	28.8394	30.8394	40	8.8394	0	SUCCESS
12	T1, J4	4	30.8394	34.1191	40	0.8394	0	SUCCESS
13	T2, J4	4	34.1191	37.1191	40	4.1191	0	SUCCESS
14	T7, J1	1	37.1191	40.1191	80	37.1191	0	SUCCESS
15	T1, J5	5	40.1191	43.3989	50	0.1191	0	SUCCESS
16	T2, J5	5	43.3989	46.3989	50	3.3989	0	SUCCESS
17	T3, J3	3	46.3989	48.3989	60	6.3989	0	SUCCESS
18	T4, J3	3	48.3989	50.3989	60	8.3989	0	SUCCESS
19	T1, J6	6	50.3989	53.6787	60	0.3989	0	SUCCESS
20	T2, J6	6	53.6787	56.6787	60	3.6787	0	SUCCESS
21	T5, J2	2	56.6787	58.6787	80	16.6787	0	SUCCESS
22	T6, J2	2	58.6787	60.6787	80	18.6787	0	SUCCESS
23	T1, J7	7	60.6787	63.9585	70	0.6787	0	SUCCESS
24	T2, J7	7	63.9585	66.9585	70	3.9585	0	SUCCESS
25	T3, J4	4	66.9585	68.9585	80	6.9585	0	SUCCESS
26	T4, J4	4	68.9585	70.9585	80	8.9585	0	SUCCESS
27	T1, J8	8	70.9585	74.2383	80	0.9585	0	SUCCESS
28	T2, J8	8	74.2383	77.2383	80	4.2383	0	SUCCESS

Figure 2: Scheduling results with all tasks on time

As shown in Figure 2, the system is fully schedulable. Despite the high execution time of τ_1 ($C_1 \approx 3.28$ ms), all 29 jobs meet their deadlines. The total performance metrics for this scenario are:

- **Total Waiting Time:** 202.4552 ms
- **Total Idle Time:** 0 ms

3.2 Scenario 2: τ_5 is allowed to miss its deadline

In accordance with the project requirements, a second test was conducted where τ_5 is deprioritized to minimize the potential interference with other tasks. In this version, τ_5 only executes if no other task is ready.

Jobs generated for the hyperperiod of 80 ms : 29 jobs

Task	Instance	Start	End	Deadline	Wait_Time	idle_Time	Status
T1, J1	1	0.0000	3.2798	10	0.0000	0	SUCCESS
T2, J1	1	3.2798	6.2798	10	3.2798	0	SUCCESS
T3, J1	1	6.2798	8.2798	20	6.2798	0	SUCCESS
T4, J1	1	8.2798	10.2798	20	8.2798	0	SUCCESS
T1, J2	2	10.2798	13.5596	20	0.2798	0	SUCCESS
T2, J2	2	13.5596	16.5596	20	3.5596	0	SUCCESS
T6, J1	1	16.5596	18.5596	40	16.5596	0	SUCCESS
T7, J1	1	18.5596	21.5596	80	18.5596	0	SUCCESS
T1, J3	3	21.5596	24.8394	30	1.5596	0	SUCCESS
T2, J3	3	24.8394	27.8394	30	4.8394	0	SUCCESS
T3, J2	2	27.8394	29.8394	40	7.8394	0	SUCCESS
T4, J2	2	29.8394	31.8394	40	9.8394	0	SUCCESS
T1, J4	4	31.8394	35.1191	40	1.8394	0	SUCCESS
T2, J4	4	35.1191	38.1191	40	5.1191	0	SUCCESS
T5, J1	1	38.1191	40.1191	40	38.1191	0	MISSED DEADLINE
T1, J5	5	40.1191	43.3989	50	0.1191	0	SUCCESS
T2, J5	5	43.3989	46.3989	50	3.3989	0	SUCCESS
T3, J3	3	46.3989	48.3989	60	6.3989	0	SUCCESS
T4, J3	3	48.3989	50.3989	60	8.3989	0	SUCCESS
T1, J6	6	50.3989	53.6787	60	0.3989	0	SUCCESS
T2, J6	6	53.6787	56.6787	60	3.6787	0	SUCCESS
T6, J2	2	56.6787	58.6787	80	16.6787	0	SUCCESS
T5, J2	2	58.6787	60.6787	80	18.6787	0	SUCCESS
T1, J7	7	60.6787	63.9585	70	0.6787	0	SUCCESS
T2, J7	7	63.9585	66.9585	70	3.9585	0	SUCCESS
T3, J4	4	66.9585	68.9585	80	6.9585	0	SUCCESS
T4, J4	4	68.9585	70.9585	80	8.9585	0	SUCCESS
T1, J8	8	70.9585	74.2383	80	0.9585	0	SUCCESS
T2, J8	8	74.2383	77.2383	80	4.2383	0	SUCCESS

Figure 3: Scheduling results with τ_5 missing its deadline.

The results in Figure 3 show that while critical tasks are protected, τ_5, J_1 effectively misses its deadline (Start: 38.11 ms, End: 40.11 ms, Deadline: 40 ms). The performance metrics for this scenario are:

- **Total Waiting Time:** 209.4552 ms
- **Total Idle Time:** 0 ms

3.3 Response time analysis results

To formally validate the schedulability of both scenarios, we analyzed the Response Time (R_{ij}^n) for each of the 29 jobs. This analysis serves to verify the mathematical model against the experimental results obtained from the simulation.

```
--- Response Time analysis (Rij) ---
  Task  R_ij (Response Time)  Max Allowed (T_i) Schedulable?
T1, J1                3.2798                10           OK
T2, J1                6.2798                10           OK
T3, J1                8.2798                20           OK
T4, J1               10.2798                20           OK
T1, J2               13.5596                20           OK
T2, J2               16.5596                20           OK
T5, J1               18.5596                40           OK
T6, J1               10.5596                30           OK
T1, J3               13.8394                20           OK
T2, J3                6.8394                10           OK
T3, J2                8.8394                20           OK
T4, J2               10.8394                20           OK
T1, J4               14.1191                20           OK
T2, J4                7.1191                10           OK
T7, J1               10.1191                50           OK
T1, J5                3.3989                10           OK
T2, J5                6.3989                10           OK
T3, J3                8.3989                20           OK
T4, J3               10.3989                20           OK
T1, J6               13.6787                20           OK
T2, J6               16.6787                20           OK
T5, J2                8.6787                30           OK
T6, J2               10.6787                30           OK
T1, J7                3.9585                10           OK
T2, J7                6.9585                10           OK
T3, J4                8.9585                20           OK
T4, J4               10.9585                20           OK
T1, J8                4.2383                10           OK
T2, J8                7.2383                10           OK
```

Figure 4: Response time for scenario 1 (on time)

The response times for all tasks remain below their respective periods ($R_{ij}^n \leq T_i$). The system is stable, and the "OK" status in the verification table confirms that every job completes before the arrival of its next instance.

```
--- Response Time analysis (Rij) ---
  Task  R_ij (Response Time)  Max Allowed (T_i) Schedulable?
T1, J1               3.2798                 10           OK
T2, J1               6.2798                 10           OK
T3, J1               8.2798                 20           OK
T4, J1              10.2798                 20           OK
T1, J2              13.5596                 20           OK
T2, J2              16.5596                 20           OK
T6, J1              18.5596                 40           OK
T7, J1              11.5596                 70           OK
T1, J3              14.8394                 20           OK
T2, J3               7.8394                 10           OK
T3, J2               9.8394                 20           OK
T4, J2              11.8394                 20           OK
T1, J4              15.1191                 20           OK
T2, J4               8.1191                 10           OK
T5, J1              10.1191                 10         LATE
T1, J5               3.3989                 10           OK
T2, J5               6.3989                 10           OK
T3, J3               8.3989                 20           OK
T4, J3              10.3989                 20           OK
T1, J6              13.6787                 20           OK
T2, J6              16.6787                 20           OK
T6, J2               8.6787                 30           OK
T5, J2              10.6787                 30           OK
T1, J7               3.9585                 10           OK
T2, J7               6.9585                 10           OK
T3, J4               8.9585                 20           OK
T4, J4              10.9585                 20           OK
T1, J8               4.2383                 10           OK
T2, J8               7.2383                 10           OK
```

Figure 5: Response time for scenario 2 (missed deadline)

In this case, the response time for τ_5, J_1 (first instance) exceeds the allowed threshold. As seen in Figure 5, the status "LATE" is triggered. This confirms that by deprioritizing τ_5, we have deliberately allowed its response time to violate the schedulability condition.

3.4 Comparative analysis

By comparing the two scenarios, we observe a counter-intuitive result: allowing τ_5 to miss its deadline actually increases the total waiting time by approximately 7 ms.

This phenomenon is explained by the accumulation of delay for τ_5. Because it is forced

to wait for all other ready tasks, its individual waiting time grows significantly, which outweighs the minor timing gains made by the other tasks. Furthermore, the `Total Idle Time` remains at 0 ms in both cases because the total workload (C_{total}) is constant over the 80 ms hyperperiod, and the processor is heavily loaded ($U \approx 0.96$).
In conclusion, the Standard NP-EDF approach is the most efficient for this specific task set, as it ensures 100% schedulability while maintaining a lower total latency.

4 Computational complexity

The computational complexity of the algorithm is a function of the number of tasks n and the number of jobs N generated within the hyperperiod.

- **Job generation:** $O(N)$, where $N = \sum(Hyperperiod/T_i)$.
- **Scheduling loop:** For each scheduling decision, the algorithm iterates through the list of ready jobs to find the minimum deadline. In the worst case, this results in a complexity of $O(N^2)$ for the entire hyperperiod.

Given the small number of tasks ($n = 7$) and the short hyperperiod, this complexity remains well within the performance limits of the WSL execution environment.

5 Conclusion

The objective of this project was to design and analyze a non-preemptive scheduler for a high-utilization task set. By implementing a hybrid measurement tool, we successfully determined a reliable WCET for τ_1 ($C_1 \approx 3.28$ ms), which placed our total processor utilization at approximately 96.5%.

Our experimental results led to several key findings:

- **System Viability:** Despite the very high CPU load, the standard NP-EDF algorithm proved that the system is 100% schedulable, with every task instance meeting its deadline.
- **Optimization:** The comparison between the two scenarios revealed a counter-intuitive outcome. Deliberately allowing τ_5 to miss its deadline did not improve the system's performance. Instead, it increased the total waiting time by 7 ms, proving that the standard EDF priority logic remains the most efficient for this specific configuration.
- **Stability:** The use of C-compiled libraries for intensive calculations significantly improved the stability of execution times.

In conclusion, this study highlights the importance of precise WCET estimation and the robustness of the EDF logic. The project successfully demonstrates that even a near-saturated system can remain reliable if the scheduling parameters are correctly tuned and verified through rigorous response time analysis.